CrowdFace 머리 탐지 학습 결과 보고서

군중 밀집도 기반 인파 사고 조기 경보 · 추가 기능 Y① | 202104183 김현중, 202204201 김태섭 | 2026-11-14

1. 한눈에 보기

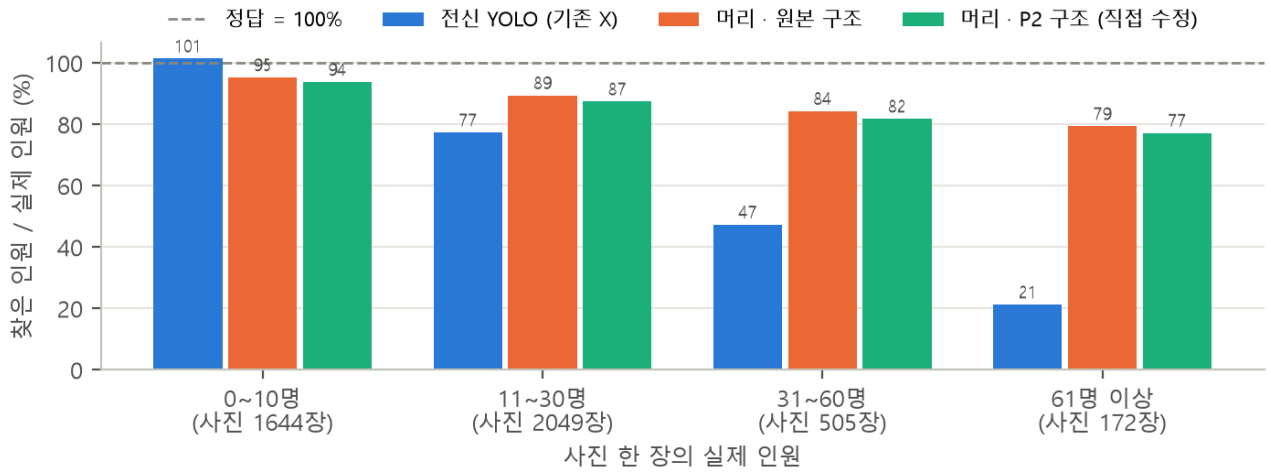
기존 YOLO는 사람 **전신**을 찾는다. 우리는 같은 YOLO 학습 코드를 이해하고 **데이터·출력 클래스·모델 구조**를 바꿔 **머리**를 찾도록 직접 학습시켰다. CrowdHuman 검증 사진 4,370장(실제 인원 85,093명)에서 인원을 세어 비교했고, 학습에 쓰지 않은 **현장 영상(HT21-03, 시부야 교차로)**에서도 확인했다.

모델	머리 mAP50	머리 mAP50-95	인원 오차 (MAE, 명/장)	찾은 비율 (전체)	찾은 비율 (61명 이상 사진)	속도 (ms/장)
전신 YOLO (기존 X)	-	-	8.15	62.0%	21.3%	4.9
머리 · 원본 구조	0.812	0.504	3.10	87.0%	79.4%	4.8
머리 · P2 구조 (직접 수정)	0.828	0.516	3.40	84.9%	77.2%	6.8

mAP: 머리 박스 위치까지 맞혔는지(검증 4,370장, IoU 기준). 인원 오차: 사진마다 |센 인원 - 실제 인원|의 평균. 찾은 비율: 센 인원 합 ÷ 실제 인원 합(100%에 가까울수록 좋음). 속도: RTX 4070, 입력 960, 추론만(검증 300장 × 3회, 세 모델 번갈아 측정한 중앙값). 이 표는 Ultralytics 기본 신뢰도 기준 0.25로 센 값(인원 세기용 기준 0.15 결과는 4-4절).

핵심 결과

- 인원 오차(MAE): 전신 8.1명 → 머리 3.1명 (**62% 감소**, 머리 · 원본 구조).
- 61명 이상 붐비는 사진: 전신은 실제 인원의 21.3%만 셌지만, 머리는 79.4%를 셌다. **붐빌수록 머리가 필요하다**는 근거.
- 직접 수정한 P2 구조: mAP50 0.812 → 0.828, mAP50-95 0.504 → 0.516로 **박스 정확도는 올랐지만**, 인원 세기(MAE 3.10 → 3.40)와 16px 미만 머리 찾기는 **나아지지 않았고** 속도는 4.8 → 6.8 ms/장으로 느려졌다 (4-2 참고).
- 단, 인원 세기용 신뢰도 기준 0.15로 맞추면 **P2가 조금 더 정확하게 센다** (MAE base 2.28 · P2 2.16, 4-4 참고).
- 현장 영상(위에서 내려다본 교차로, 화면당 평균 257명): 전신은 6.2%~27.4%만 찾고 붐빌수록 밀도를 **더 낮게** 봤다(상관 -0.38). 머리는 76.7%를 찾고 붐비는 흐름을 따라갔다(5절).



검증 사진을 실제 인원으로 묶어, 각 모델이 센 인원 ÷ 실제 인원을 비교했다. 100%보다 낮으면 덜 센 것.

2. 이전 코드와 무엇이 달라졌나

	이전 (기존 기능 X)	현재 (추가 기능 Y①)
찾는 대상	사람 전신	사람 머리 (모자·뒤통수 포함)
학습 데이터	COCO (Ultralytics가 학습해 배포)	CrowdHuman 15,000장 · 머리 287,106개 (직접 변환)
출력 클래스	COCO 80종 중 person 사용	head 1종 (Detect 층 출력 80 → 1)
모델 구조	YOLO11n 원본 (P3·P4·P5에서 탐지)	① base: 원본 그대로 ② P2: 작은 머리용 P2 탐지층 추가 (직접 수정)
시작 가중치	COCO 학습 결과	COCO 가중치에서 전이학습 (맞는 층만 이어받음)
코드	src/yolo_person.py	src/convert_crowdhuman.py · models/head_p2.yaml · src/train_head.py · src/eval_heads.py

Ultralytics 라이브러리 코드는 고치지 않았다. 바꾼 것은 **무엇을 가르치는지(데이터·클래스)**와 **모델 구조 설정 파일**이다.

2-1. 데이터 변환: CrowdHuman → YOLO 머리 라벨

CrowdHuman 라벨(.odgt)은 사진 한 장마다 사람 목록이 있고, 사람마다 전신·보이는 부분·머리 박스가 있다. 우리는 머리 박스(hbox)만 골라 YOLO 형식 "0 cx cy w h"(0~1 비율)로 바꿨다.

- **mask 영역**(포스터·마네킹, 또는 너무 빽빽해 라벨을 못 단 군중)과 **ignore 표시된 머리**는 학습에서 뺐다.
- 그냥 빼면 그 안의 진짜 머리가 '머리 아님'으로 학습된다. Ultralytics에는 무시 영역 기능이 없으므로 **그 영역을 회색(114)으로 칠해 사진에서 지웠다**. (학습 105,490곳, 검증 30,000곳)
- 사진 밖으로 나간 박스는 경계에 맞춰 자르고, 2픽셀보다 작으면 뺐다.
- 통계: 머리의 약 30%가 높이 16픽셀 미만 → 작은 머리를 잘 찾는 구조가 필요하다는 근거.

2-2. 모델 구조 변경: P2 탐지층 추가 (models/head_p2.yaml)

원본 YOLO11n은 1/8·1/16·1/32 해상도(P3·P4·P5) 세 곳에서 탐지한다. 16픽셀 머리는 P3 특징 지도에서 2칸뿐이라 놓치기 쉽다. 그래서 백본 2번 층의 **1/4 해상도(P2) 특징을 끌어와** 한 단계 더 촘촘한 곳에서도 탐지하게 했다. Ultralytics에는 YOLO11용 P2 설정이 없어(YOLOv8용만 있음) 원본 설정을 직접 고쳤다.

원본 yolo11.yaml (head 부분)

```
- [-1, 1, nn.Upsample, [None, 2, "n"]]
- [[-1, 6], 1, Concat, [1]]
- [-1, 2, C3k2, [512, False]]
- [-1, 1, nn.Upsample, [None, 2, "n"]]
- [[-1, 4], 1, Concat, [1]]
- [-1, 2, C3k2, [256, False]]
- [-1, 1, Conv, [256, 3, 2]]
- [[-1, 13], 1, Concat, [1]]
- [-1, 2, C3k2, [512, False]]
- [-1, 1, Conv, [512, 3, 2]]
- [[-1, 10], 1, Concat, [1]]
- [-1, 2, C3k2, [1024, True]]
- [[16, 19, 22], 1, Detect, [nc]]
```

직접 수정한 head_p2.yaml (head 부분, '+ 추가' 표시)

```
  # 위에서 아래로 (큰 특징 → 작은 특징), 원본과 동일
- [-1, 1, nn.Upsample, [None, 2, "n"]] # 11
- [[-1, 6], 1, Concat, [1]] # 12 cat backbone P4
- [-1, 2, C3k2, [512, False]] # 13
- [-1, 1, nn.Upsample, [None, 2, "n"]] # 14
- [[-1, 4], 1, Concat, [1]] # 15 cat backbone P3
- [-1, 2, C3k2, [256, False]] # 16 (P3/8)
  # + 추가: P2까지 한 단계 더 내려감
- [-1, 1, nn.Upsample, [None, 2, "n"]] # 17
- [[-1, 2], 1, Concat, [1]] # 18 cat backbone P2
- [-1, 2, C3k2, [128, False]] # 19 (P2/4-xsmall) ← 새 탐지 위치
  # 아래에서 위로 다시 올라가며 합침 (P2 → P3 → P4 → P5)
- [-1, 1, Conv, [128, 3, 2]] # 20 + 추가
- [[-1, 16], 1, Concat, [1]] # 21 + 추가: cat P3
- [-1, 2, C3k2, [256, False]] # 22 (P3/8-small)
- [-1, 1, Conv, [256, 3, 2]] # 23
- [[-1, 13], 1, Concat, [1]] # 24 cat P4
- [-1, 2, C3k2, [512, False]] # 25 (P4/16-medium)
- [-1, 1, Conv, [512, 3, 2]] # 26
- [[-1, 10], 1, Concat, [1]] # 27 cat P5
- [-1, 2, C3k2, [1024, True]] # 28 (P5/32-large)
- [[19, 22, 25, 28], 1, Detect, [nc]] # Detect(P2, P3, P4, P5)
```

	원본 구조 (base)	P2 구조
탐지 위치(stride)	8 · 16 · 32	4 · 8 · 16 · 32
파라미터 수	2,604,883	2,666,820
연산량 (GFLOPs, 640 기준)	6.6	10.5
COCO에서 이어받은 가중치	493/499	297/593

P2 구조는 파라미터는 거의 같지만, 큰 해상도에서 계산하므로 연산량이 늘어난다.

3. 어떻게 학습했나

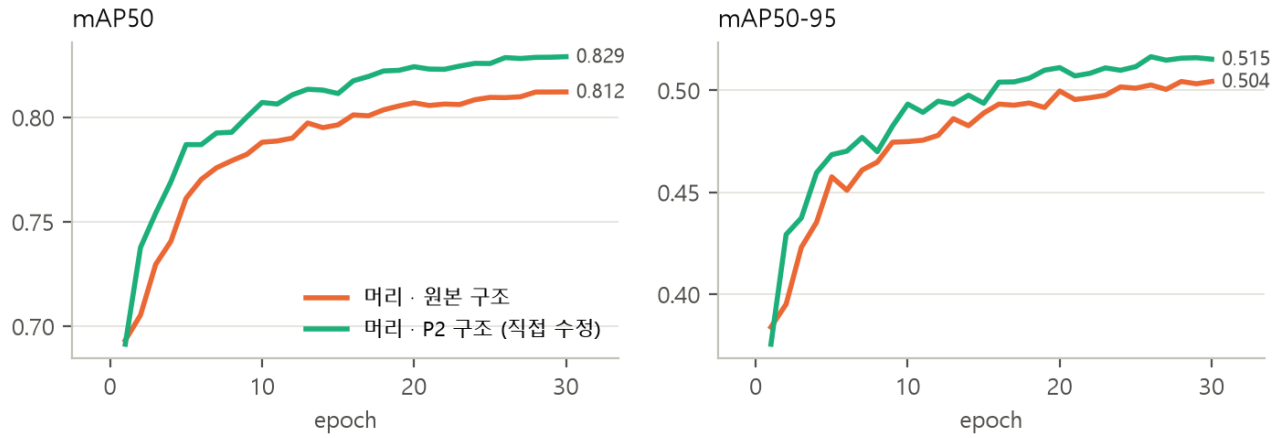
항목	값	이유
데이터	학습 15,000장 / 검증 4,370장 (CrowdHuman 공식 분할)	검증 사진은 학습에 쓰지 않음
입력 크기	960	머리가 작아 기본값 640보다 크게
epoch / batch	30 / base 16 · P2 8	P2는 batch 16에서 GPU 메모리(12GB)가 모자라 8로 줄임. Ultralytics는 기울기를 모아 유효 batch 64로 맞추므로(16×4 = 8×8) 학습률·최적화 조건은 같음
최적화	AdamW (lr 0.002)	Ultralytics 자동 선택
장비	RTX 4070 (12GB), Python 3.13, torch 2.15 + CUDA	Colab 대신 이 PC GPU
학습 시간	base 1.3시간 · P2 5.7시간	P2는 22 epoch에서 한 번 멈췄다가 resume으로 이어서 학습 (밤에는 다른 작업과 GPU를 나눠 써서 더 느렸음)
가장 좋은 epoch	base 30 · P2 26	검증 점수가 가장 높은 가중치를 best.pt로 저장

학습 명령 (같은 Ultralytics 학습 코드 model.train)

```
# base: COCO 가중치를 불러오면 Detect 층 출력만 1클래스로 새로 만들어짐
YOLO('yolo11n.pt').train(data='head.yaml', epochs=30, imgsz=960, batch=16)

# P2: 직접 수정한 구조를 만든 뒤, 모양이 맞는 층만 COCO 가중치를 이식
YOLO('head_p2.yaml').load('yolo11n.pt').train(data='head.yaml', epochs=30, imgsz=960, batch=8)
```

학습 곡선 (검증 사진 점수, epoch마다)



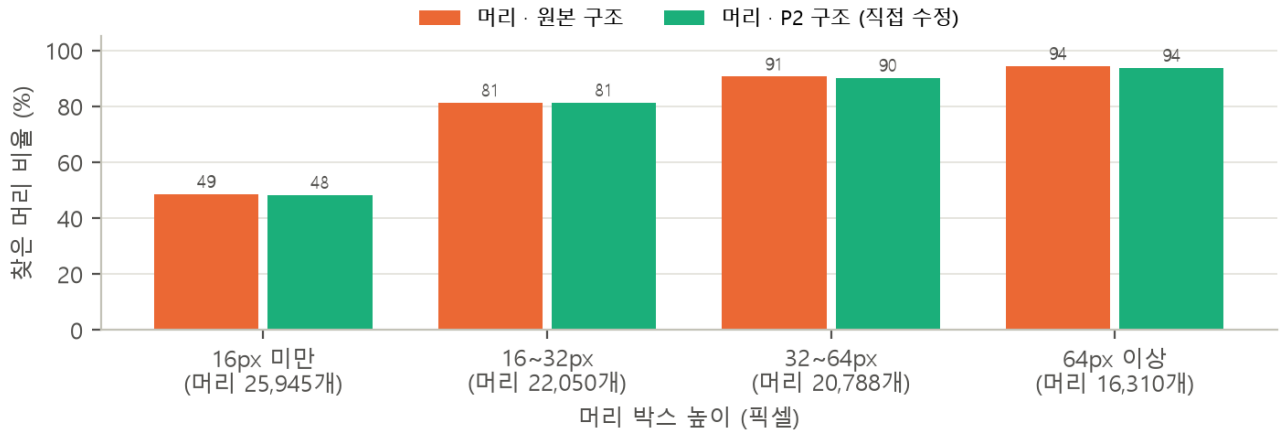
선이 오르다 평평해지면 더 배울 것이 적다는 뜻. 끝의 숫자는 마지막 epoch 점수.

4. 결과

4-1. 사람이 많을수록 전신 YOLO는 덜 센다 (1쪽 그림)

- 전신 YOLO: 101.4% → 77.3% → 47.3% → 21.3% (사람이 많아질수록 몸이 가려져 박스를 놓치고, 겹친 박스는 하나로 합쳐짐).
- 머리 · 원본 구조: 95.3% → 89.4% → 84.3% → 79.4% (머리는 가려지는 일이 적어 붐벼도 비율이 유지됨).
- 머리 · P2 구조 (직접 수정): 93.8% → 87.5% → 81.8% → 77.2% (머리는 가려지는 일이 적어 붐벼도 비율이 유지됨).

4-2. 머리 크기별로 찾은 비율 (P2 구조의 효과)



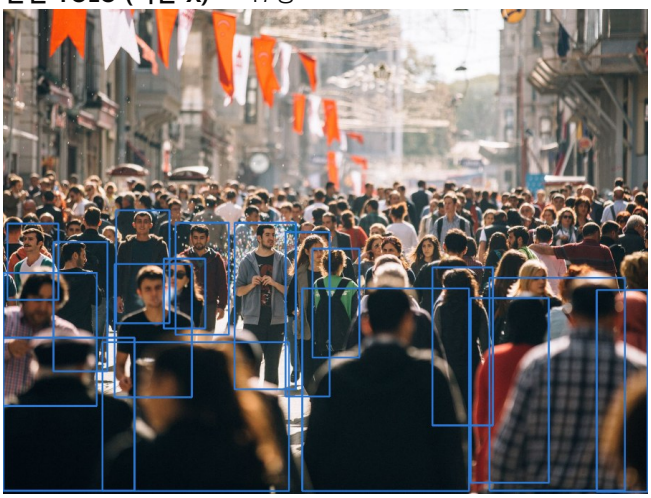
정답 머리와 IoU 0.5 이상 겹치는 예측이 있으면 '찾음'. 작은 머리에서 두 구조의 차이를 본다.

- 16px 미만: base 48.7% → P2 48.4% (-0.3%p)
- 16~32px: base 81.3% → P2 81.4% (+0.1%p)
- 32~64px: base 90.7% → P2 90.2% (-0.4%p)
- 64px 이상: base 94.4% → P2 93.8% (-0.6%p)
- **해석**: 모든 크기에서 차이가 1%p 이내다. P2를 넣어도 16px 미만 머리는 여전히 절반 정도만 찾았다. P2의 mAP 상승(mAP50-95 +1.2%p)은 '더 많이 찾기'가 아니라 **박스 위치를 더 정확히 맞춘 것**에서 나왔다(mAP50-95는 IoU 0.5~0.95를 평균해 위치 정확도를 더 많이 반영).
- **작은 머리가 안 잡히는 이유(추정)**: 입력 960으로 줄이면서 원본에서 이미 작던 머리가 몇 픽셀까지 작아진다. 탐지층을 늘려도 입력에 정보가 없으면 찾을 수 없다. 그래서 구조 대신 입력을 바꾸는 타일링을 직접 구현해 확인했다(4-5절). 신뢰도 기준 맞추기는 4-4절.

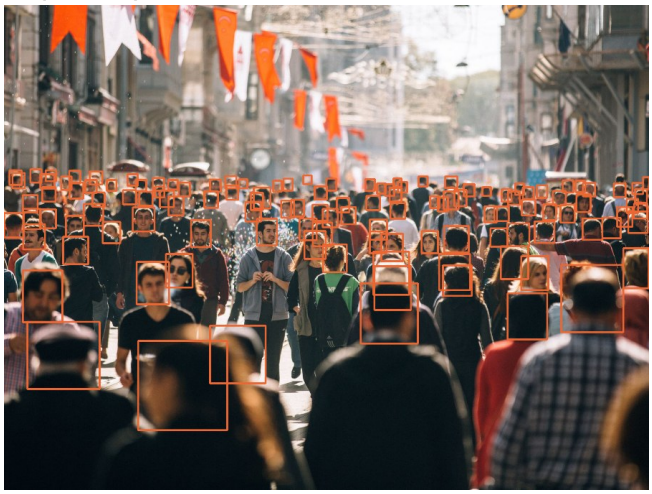
4-3. 같은 사진, 모델별 결과

DM-Count 예시 사진(학습에 쓰지 않은 사진). 박스 색은 모델 색과 같다.

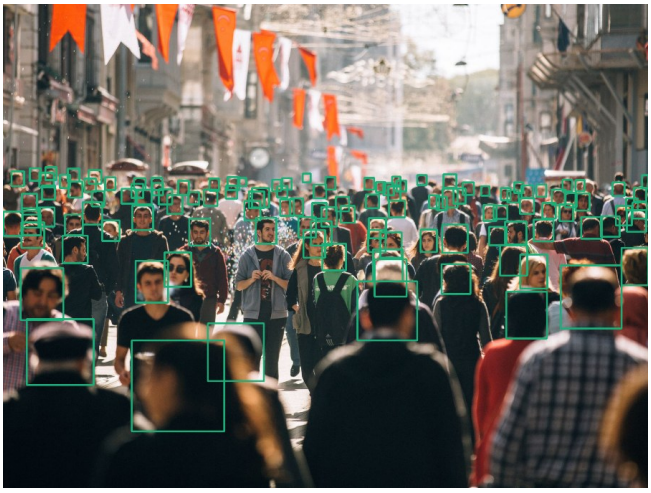
전신 YOLO (기존 X) — 17명



머리 · 원본 구조 — 123명



머리 · P2 구조 (직접 수정) — 122명



4-4. 인원 세기용 신뢰도 기준 맞추기 (0.25 → 0.15)

모델은 박스마다 '머리일 확신'(0~1)을 내고, 기준 이상인 박스만 센다. Ultralytics 기본값 0.25는 mAP 계산용이라 인원을 덜 세는 쪽(87%)이었다. 그래서 **검증 사진에서 센 인원 합 = 실제 인원 합**이 되는 기준을 찾았더니(src/calib_conf.py) 0.15였다.

모델	기준	인원 오차 (MAE)	찾은 비율 (전체)	찾은 비율 (61명 이상)
전신 YOLO (기존 X)	0.25	8.15	62.0%	21.3%
	0.15	6.79	74.7%	30.3%
머리 · 원본 구조	0.25	3.10	87.0%	79.4%
	0.15	2.28	100.4%	98.1%
머리 · P2 구조 (직접 수정)	0.25	3.40	84.9%	77.2%
	0.15	2.16	99.8%	99.7%

같은 검증 4,370장. 전신 YOLO도 같은 0.15로 다시 셌다(공정 비교). 기준을 같은 검증 사진으로 정했으므로 이 표의 0.15 결과는 약간 유리하게 나온 값이다(현장 영상 5절은 따로 정한 기준).

- 머리 · 원본 구조: MAE 3.10 → **2.28**, 찾은 비율 87.0% → 100.4%
- 머리 · P2 구조 (직접 수정): MAE 3.40 → **2.16**, 찾은 비율 84.9% → 99.8%
- **P2 결론이 바뀐다**: 기본 기준(0.25)에서는 머리 · 원본 구조의 인원 오차가 작았지만, 기준을 맞추면 **머리 · P2 구조 (직접 수정)**가 더 정확하다. 0.25에서는 덜 세는 손해를 더 크게 봤던 것. 모델 비교는 같은 '세기 조건'에서 해야 한다는 교훈.
- 전신 YOLO는 기준을 낮춰도 74.7%, 61명 이상 사진에서 30.3%에 그친다. 가려진 몸은 기준을 낮춰도 박스 자체가 잘 나오지 않는다.

4-5. 타일링: 사진을 조각내 확대해서 탐지 (src/tile_detect.py, 직접 구현)

4-2에서 P2 탐지층을 넣어도 작은 머리 재현율은 그대로였다. 원인을 '입력을 960으로 줄이며 작은 머리 정보가 사라짐'으로 추정했으므로, 모델·가중치는 그대로 두고 **입력을 바꿨다**.

- 사진을 n×n 조각으로 나눠(이웃과 20% 겹침) 조각마다 960 크기로 탐지 → 같은 머리가 더 크게 들어간다.
- 조각의 안쪽 경계에 닿은 박스는 잘린 머리이므로 버리고, 조각보다 큰 머리를 위해 전체 사진 탐지도 한 번 더한다.
- 여러 조각·전체 탐지에서 중복으로 잡힌 머리는 겹친 박스 합치기(NMS, IoU 0.5)를 직접 짜서 하나만 남긴다.

방식	신뢰도 기준	인원 오차 (MAE)	맞힌 머리 (재현율)	16px 미만	16~32px	32px 이상	시간 (ms/장)
보통 탐지 (전체 사진 1번)	0.15	2.28	79.2%	54.8%	84.1%	93.4%	22
타일링 2×2 + 전체	0.25	1.99	83.6%	63.4%	89.0%	94.6%	59
타일링 3×3 + 전체	0.27	2.19	84.1%	64.2%	89.7%	94.6%	86

CrowdHuman 검증 4,370장, 머리 원본 구조(head_base). 타일링은 박스를 더 많이 내므로 방식마다 '센 인원 합 = 실제 인원 합'이 되는 기준을 따로 맞춘 뒤 비교했다(같은 개수를 셀 때 진짜 머리를 얼마나 맞혔는지). 시간은 사진 읽기 포함이라 1쪽 표의 속도(추론만)와 다르다.

- **작은 머리가 잡힌다**: 16px 미만 재현율 54.8% → **63.4%**, 전체 재현율 79.2% → 83.6%, 인원 오차 2.28 → 1.99. P2(구조 변경)로 안 되던 것이 입력 변경으로 됐다 → **원인은 탐지층이 아니라 입력 해상도**.
- 2×2면 충분하다. 3×3은 재현율이 거의 같고 시간만 늘어난다(59 → 86 ms/장). 대가는 속도: 보통 탐지의 약 2.7배.
- **현장 영상(HT21-03)에서는 효과가 없었다**: 찾은 비율(보통 → 2×2 → 3×3)이 원본 구조 72.1% → 70.2% → 66.7%, 현장 추가 학습 73.3% → 74.1% → 67.4%. 이 영상의 머리는 높이 26~45px(중앙 33px)로 작지 않다. 놓치는 원인이 크기가 아니라 모양(검은 뒤통수 + 검은 외투)이라 확대해도 소용없고, 3×3은 너무 확대돼 헛박스가 늘었다.
- 결론: 타일링은 머리가 작게 찍히는 먼 카메라용 도구다. 시연 영상은 보통 탐지를 그대로 쓴다. (이 실험은 기준을 0.01 단위로 찾아 head_ht21 기준이 0.22이고, 5절은 0.05 단위로 찾은 0.20이라 찾은 비율이 조금 다르다.)

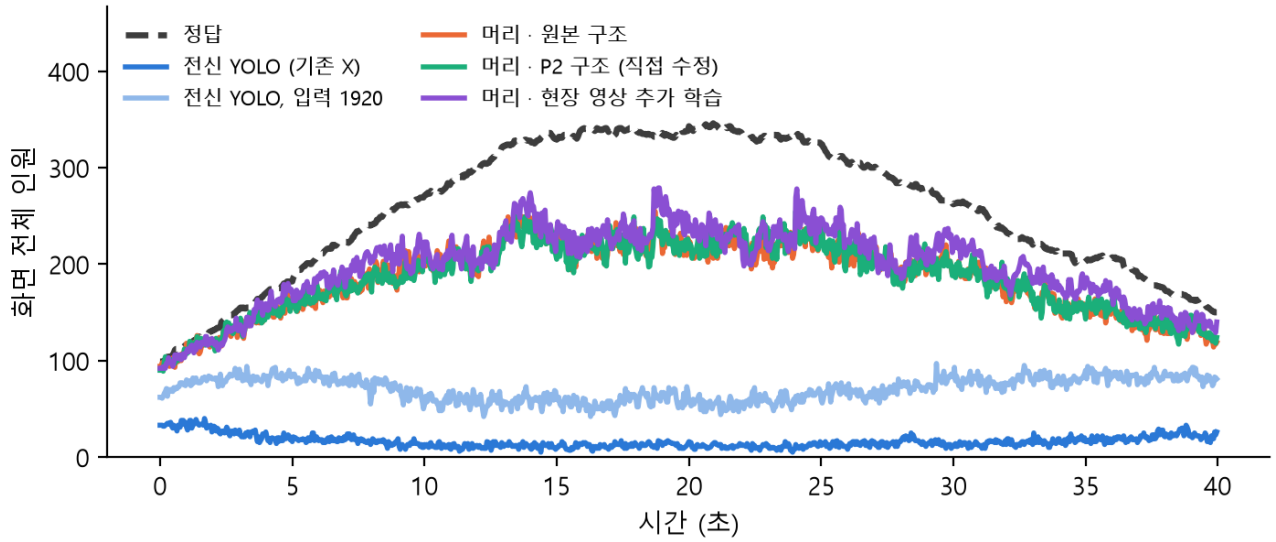
5. 현장 영상 평가 (HT21-03, 학습에 쓰지 않은 영상)

CrowdHuman은 대부분 눈높이 사진이다. 실제 CCTV처럼 **위에서 내려다본 영상**에서도 되는지 HT21(CroHD, MOTChallenge 머리 추적 데이터셋, CC BY-NC-SA 3.0)의 HT21-03(시부야 스크램블 교차로, 1000프레임 = 40초, 화면당 평균 257명)으로 평가했다. 모든 프레임에 머리 박스 정답이 있다.

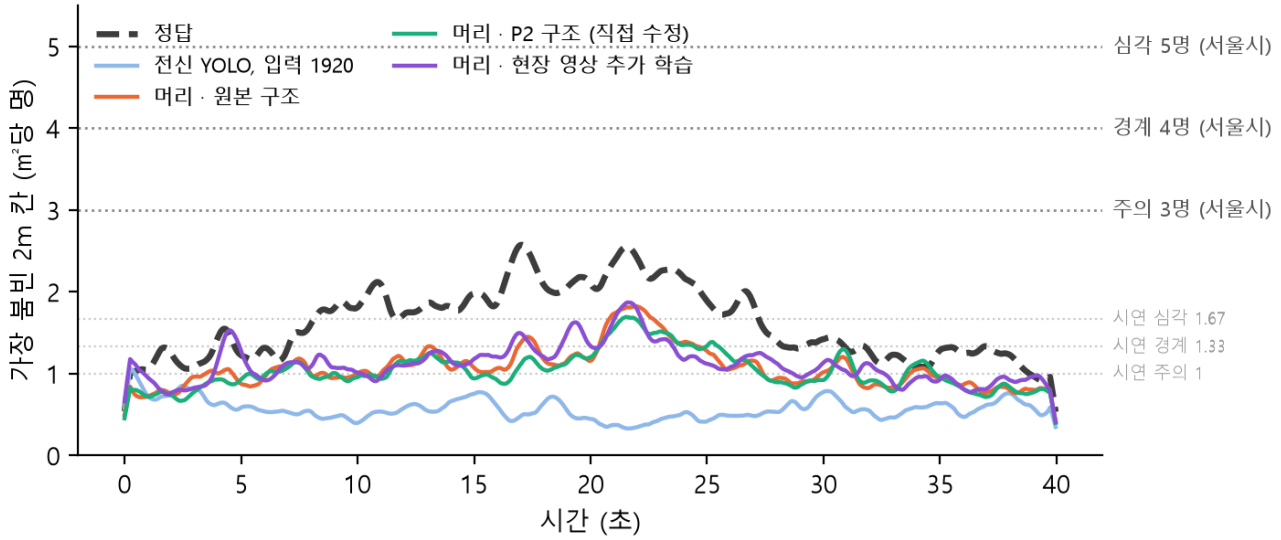
- 바닥: 화면의 네 점 → 10m × 28m 직사각형으로 원근 변환(호모그래피). 넓이는 횡단보도 줄 간격 0.9m 가정 + 맨홀 타원 비율로 추정.
- 밀도: 머리 박스 아래로 머리 높이×6 지점을 발 위치로 보고 바닥 좌표로 옮긴 뒤, 2m×2m 칸마다 ㎡당 인원. 칸별로 1초 평균(2m 칸은 1명 차이가 ㎡당 0.25명이라 프레임마다 출렁임).
- 정답(gt.txt)의 머리 박스도 똑같은 방법으로 밀도를 계산해 비교했다.
- head_ht21: head_base를 HT21-01·02·04(같은 데이터셋의 **다른** 장소)로 20 epoch 추가 학습(5분). HT21-03은 학습에도, 신뢰도 기준(0.20, HT21-02 뒷부분으로 정함)에도 쓰지 않았다.

모델	신뢰도 기준	찾은 비율	인원 오차 (명/프레임)	최대 칸 밀도 치우침 (명/㎡)	밀도 흐름 일치 (상관계수)
전신 YOLO (입력 960)	0.25	6.2%	241	-1.30	**-0.38**
전신 YOLO (입력 1920)	0.25	27.4%	187	-1.05	**-0.51**
머리 · 원본 구조	0.15	72.1%	72	-0.55	0.79
머리 · P2 구조	0.15	72.0%	72	-0.58	0.72
머리 · 현장 영상 추가 학습	0.20	76.7%	60	-0.50	0.75

찾은 비율: 센 인원 합 ÷ 정답 인원 합. 최대 칸 밀도 치우침: 가장 붐비는 칸의 밀도(모델 - 정답)의 평균, 음수면 덜 붐비게 봄. 밀도 흐름 일치: 40초 동안 '가장 붐비는 칸 밀도' 곡선이 정답 곡선과 같이 오르내리는 정도(1이면 완전히 같고, 음수면 반대로 움직임).



프레임마다 센 인원. 사람들이 횡단보도로 몰려들수록 머리 모델은 따라 늘지만 전신은 오히려 줄어든다.



가장 붐비는 2m 칸의 ㎡당 인원(1초 평균). 가로줄은 시연용 경보 기준.

결과 해석

- **전신은 붐빌수록 반대로 본다**: 상관 -0.38(입력 960) / -0.51(입력 1920). 위에서 내려다보면 몸이 서로 가려 사람이 몰릴수록 박스가 줄어든다. 입력을 키워도 찾은 비율은 27.4%뿐. **경보에 쓰면 가장 위험할 때 '괜찮다'고 한다.**
- 머리 모델은 붐비는 흐름을 따라간다(상관 0.79). 다만 처음 보는 환경이라 72.1%만 찾았고, 놓친 것은 주로 **뒤돌아선 검은 머리 + 검은 외투**(배경과 구분이 안 됨)였다.
- 현장 영상으로 5분 추가 학습(head_ht21)하니 찾은 비율 72.1% → **76.7%**, 인원 오차 72 → 60명. 같은 학습 코드로 현장에 맞출 수 있다.
- P2 구조는 이 영상에서 base와 찾은 비율이 거의 같았다(72.1% vs 72.0%).

경보 비교

- 서울시 기준(㎡당 3·4·5명): 정답 밀도가 최대 2.62명/㎡(1초 평균)라 **40초 내내 '정상'**. 모든 모델도 정상으로 판단했다(일치율 100%). 실제로 위험한 장면은 아니었다.
- 경보가 동작하는 모습을 보이기 위해 **시연용 기준**(서울시의 1/3: ㎡당 1 · 1.33 · 1.67명)으로 같은 영상을 다시 판단했다. 시연 화면에도 '시연용 기준'을 표시한다.

	시연 기준 경보 (같은 단계가 5초 이어지면 울림)
정답 머리 박스	5.0초 주의 → 12.1초 경계 → 16.8초 심각 → 27.2초 경계 → 28.3초 주의 → 38.8초 해제
전신 YOLO (입력 960)	경보 없음
전신 YOLO (입력 1920)	경보 없음
머리 · 원본 구조	21.5초 주의 → 27.4초 해제
머리 · P2 구조	21.7초 주의 → 27.4초 해제
머리 · 현장 영상 추가 학습	16.1초 주의 → 28.8초 해제

시간은 영상 시작부터 초. '해제'는 모든 칸이 기준 아래로 5초 이어진 시각.

- 전신 모델은 시연 기준에서도 **경보를 한 번도 울리지 못했다**. 머리 모델은 경보를 울리지만 정답보다 늦고 낮은 단계(주의)에 그쳤다 → 사람을 놓친 만큼 밀도를 실제의 약 0.65배로 봤기 때문.
- 시연 영상 만들기: python src/crowd_alarm.py data/ht21/HT21/train/HT21-03/img1 results/demo_ht21_03.mp4 --model head_ht21 --conf 0.2 --levels 1,1.33,1.67 (영상은 데이터 라이선스 때문에 GitHub에 올리지 않음)

6. 정리와 발표 포인트

- **기존 기능을 다르게 적용**: 같은 YOLO 학습 코드에 데이터·클래스·구조를 바꿔 '전신' 대신 '머리'를 찾게 했다 (공지 예시의 '마스크 → 썬글라스 추가'와 같은 방식).
- **직접 이해하고 바꾼 부분**: ① CrowdHuman 라벨 변환 규칙(무시 영역 회색 처리) ② Detect 층 80 → 1 클래스와 전이학습 ③ YOLO11 구조 파일에 P2 탐지층 추가.
- **왜 머리인가**: 붐비는 사진(61명 이상)에서 전신은 21.3%, 머리는 79.4%를 셌다. 인파 사고는 붐빌 때 일어나므로 이 차이가 핵심.
- **구조 수정의 효과**: P2 구조는 mAP가 올랐고(박스 위치 정확도), 인원 세기용 기준(0.15)에서는 인원 오차도 base보다 조금 작았다. 하지만 16px 미만 머리를 찾는 비율은 그대로였고 속도는 느려졌다. 즉 **작은 머리 문제는 탐지층 추가만으로는 풀리지 않았다**. 현장 영상에서는 두 구조의 찾은 비율이 같았으므로(5절) 시연에는 빠른 원본 구조와, 그것을 현장 영상으로 추가 학습한 head_ht21을 쓴다.
- **가설 → 실패 → 원인 → 해결**: 탐지층 추가(P2)로 안 풀린 작은 머리를, 사진을 조각내 확대해 넣는 타일링(직접 구현)으로 16px 미만 재현율 54.8% → 63.4%로 올렸다. 원인은 구조가 아니라 입력 해상도였다. 단, 머리가 작지 않은 현장 영상에서는 효과가 없었다(4-5절).
- **현장 영상에서도 같은 결론**: 전신은 붐빌수록 밀도를 낮게 보는 반대 방향(상관 -0.38)이라 경보에 쓸 수 없고, 머리는 흐름을 따라간다. 다만 머리도 실제보다 적게 세므로 **놓치면 위험을 낮게 본다** → 경보 기준을 정할 때 이 오차를 감안해야 한다.

한계

- 현장 영상 평가는 HT21-03 한 편(40초)이다. 바닥 넓이(10m×28m)는 횡단보도 줄 간격(0.9m 가정)과 맨홀 모양으로 추정한 값이라 ㎡당 밀도에는 추정 오차가 들어 있다(인원 비교는 영향 없음).
- 신뢰도 기준은 데이터마다 다르다(CrowdHuman 0.15, 현장 추가 학습 모델 0.20). 새 현장에서는 다시 맞춰야 한다.
- P2는 메모리 때문에 batch 8로 학습했다(base는 16). 유효 batch는 64로 같지만 BatchNorm 통계는 작은 batch로 계산된다.
- 정답 인원은 무시 영역을 뺀 머리 라벨 개수다. 회색으로 지운 영역의 사람은 정답에도, 예측에도 들어가지 않는다.

다음 할 일

- 어두운 뒤통수 찾기: 현장 영상 추가 학습을 늘리거나, 전신 박스로 보완
- 전신·머리 결합 세기 (머리 중심이 전신 박스 위쪽 1/3 안이면 같은 사람)
- 발표 자료와 시연 영상 정리 (src/crowd_alarm.py, 시연용 기준 표시 유지)